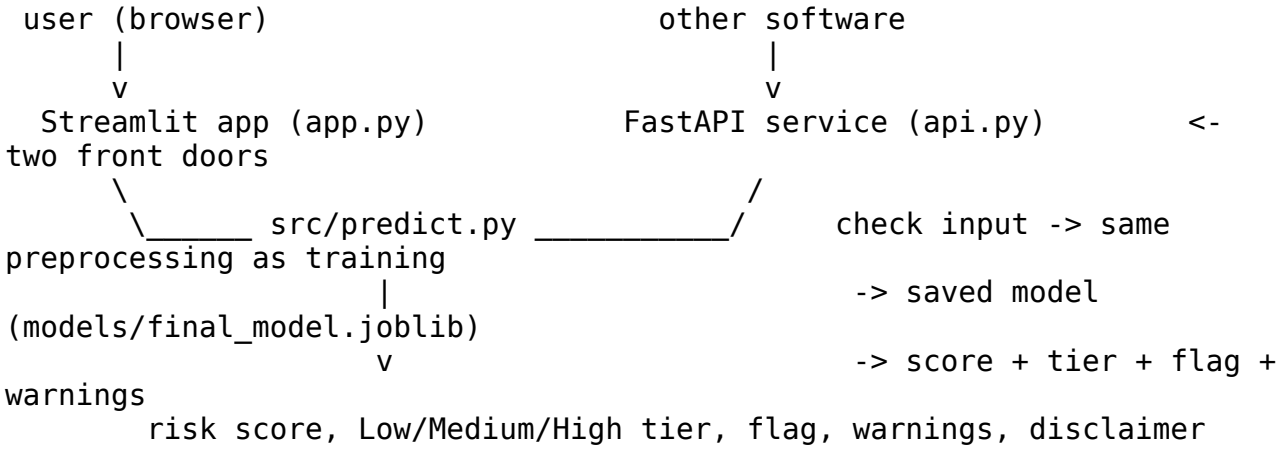
# Deployment approach, testing and experiment tracking

## 1. How the prototype fits together




Both front doors call the same function (`predict_one`), so one record gives the same score in the app, in the API and in the notebook.

## 2. Ways to run or reach it

| Way | Command or place | Status |
|---|---|---|
| **GitHub repository** | `git clone https://github.com/tsega-ab1/diabetic-readmission-risk.git` | Implemented |
| **Live web app** | Streamlit Community Cloud, link in the README | Implemented if the link in the README is filled in (it deploys straight from the GitHub repository) |
| **Local web app** | `python -m pip install -r requirements.txt` then `streamlit run app.py` | Implemented, run by group members on Linux and tested headlessly |
| **API** | `python -m pip install -r requirements-api.txt` then `python -m uvicorn api:app --port 8000`; open `http://localhost:8000/docs` | Implemented and tested (see section 4) |
| **Container** | `docker build -t readmission-risk .` then `docker run --rm -p 8501:8501 readmission-risk` (API: add `python -m uvicorn api:app --host 0.0.0.0 --port 8000` and | `Dockerfile` written; see the status note in section 5 |

| Way | Command or place | Status |
|---|---|---|
| | use `-p 8000:8000`) | |
| **Notebook** | Google Colab or Jupyter (`notebooks/module5_ai_project_colab.ipynb`) | Implemented, executed in Colab |

## 3. Inputs, outputs and error handling

**Input:** one hospital stay, as the 46 fields described in `DATA_DICTIONARY.md` and `docs/APP_GUIDE.md` (form in the app, JSON in the API). **Output:** a relative risk score (0 to 1), a Low/Medium/High tier, a true/false flag at the 0.51 threshold, a list of warnings and a disclaimer.

API example (`outputs/results/sample_typical_request.json` is a real test-set row):

```
POST /predict   {"record": { ...46 fields... }}
->  {"risk_score": 0.4857, "risk_tier": "Medium", "high_risk_flag":
false, "decision_threshold": 0.51,
     "input_warnings": [], "ignored_fields": [], "disclaimer": "Academic
demonstration only ..."}
```

| Problem with the input | What happens |
|---|---|
| A field is missing | The pipeline fills it with a typical value; the response names it in `input_warnings` |
| Text in a number field | Treated as missing and warned about; no crash |
| Number outside the range seen in training | Warned about ("prediction may be unreliable"); still scored |
| Category the model never saw | Treated as "unknown"; warned about |
| Patient ID or encounter number sent | Ignored (never used) and listed in `ignored_fields` |
| Request is not valid JSON of the right shape | API returns HTTP 422; nothing is scored |

## 4. Testing and evidence

| Level | What | Result | Evidence |
|---|---|---|---|
| Unit tests (pytest) | ICD-9 grouping, target recoding, exclusions, missing-value coding, tier boundaries, model loading, output format, missing and invalid input, identifiers not used | **16 passed** | `docs/evidence/pytest_verbose.txt`, `docs/evidence/run_tests_without_pytest.txt` |
| API tests (pytest) | health, model info, same score as the app, missing/invalid values, unseen category and extreme values, identifiers ignored, malformed request | **7 passed** (23 passed in total with the unit tests) | `docs/evidence/pytest_verbose.txt` |
| Real HTTP session | The API was run with uvicorn and called with | Scores and warnings as expected; malformed | `docs/evidence/api_session.txt` |

| Level | What | Result | Evidence |
|---|---|---|---|
| | `curl`: a normal record, a record with odd values, and a malformed request | request returned 422; the server log held no patient values | |
| End-to-end workflow cases | 7 cases through the saved model reloaded from disk: normal, missing value, invalid number, unseen category, extreme values, higher-risk, lower-risk | **All 7 passed** | Notebook Section 8.2 and report Section 12 |
| In-notebook unit tests | 10 checks of the pipeline | **10 passed** | Notebook Section 8.3 |
| Web app | The Streamlit app was run headlessly on the three sample cases (48.6% Medium, 69.0% High, 23.8% Low) and a group member ran it in a browser | Matches the notebook | Report Section 13 |
| Model evaluation | Held-out, patient-disjoint test set of 14,859 stays | ROC-AUC 0.6856, PR-AUC 0.2390, recall 0.5289; 5 of 6 success criteria met | Report Section 11, `outputs/results/notebook_results.json` |

Screenshots of the app, tests, API page and MLflow screen are in `docs/screenshots/`.

**A note on the hand-made inputs.** Edge-case inputs (a 60-day stay, race "Martian", and so on) are invented only to test error handling. See `docs/SIMULATED_DATA_STATEMENT.md`.

## 5. FastAPI and Docker: what they are for, and what we did

- **FastAPI** turns the model into a web service that other software can call (for example, a hospital system sending a discharge record and getting a score back). It also builds an interactive test page at `/docs`. We **implemented and tested** it (`api.py`, `tests/test_api.py`, a real uvicorn session).
- **Docker** packs the app, the model and the exact library versions into one container, so it runs the same on any computer or server. We wrote a `Dockerfile` (non-root user, only the model and sample cases inside, no data and no secrets) and a `.dockerignore`. **Status: the image was not built in the environment where the Dockerfile was written, so treat it as untested until a group member runs `docker build` and `docker run`.** (Update this sentence if you build it.)

## 6. Experiment tracking

Every candidate model (validation results) and the final model (test results) are recorded with their settings in **MLflow** and in a plain CSV file, so tracking works even without MLflow. See `docs/EXPERIMENT_TRACKING.md`.

## 7. Putting the web app online (Streamlit Community Cloud)

1. Make sure the GitHub repository is public and contains `app.py`, `requirements.txt`, `models/` and `outputs/results/` (it does).
2. Go to https://share.streamlit.io and sign in with GitHub.
3. Click *Create app → Deploy a public app from GitHub*.

4. Repository: `tsega-ab1/diabetic-readmission-risk`; Branch: `main`; Main file path: `app.py`.
5. Open *Advanced settings* and choose **Python 3.12**. Click *Deploy*. The first build takes a few minutes.
6. Copy the address (it ends in `.streamlit.app`) into the README.

Free apps go to sleep after inactivity. Open the link and wake it just before recording or submitting.

## 7b. What a real deployment would add

Login and HTTPS, an audit trail, a hosted API behind the hospital network, automated tests on every change, monitoring and alerting, and clinical validation. See `docs/MONITORING_SUPPORT_MAINTENANCE_PLAN.md`.